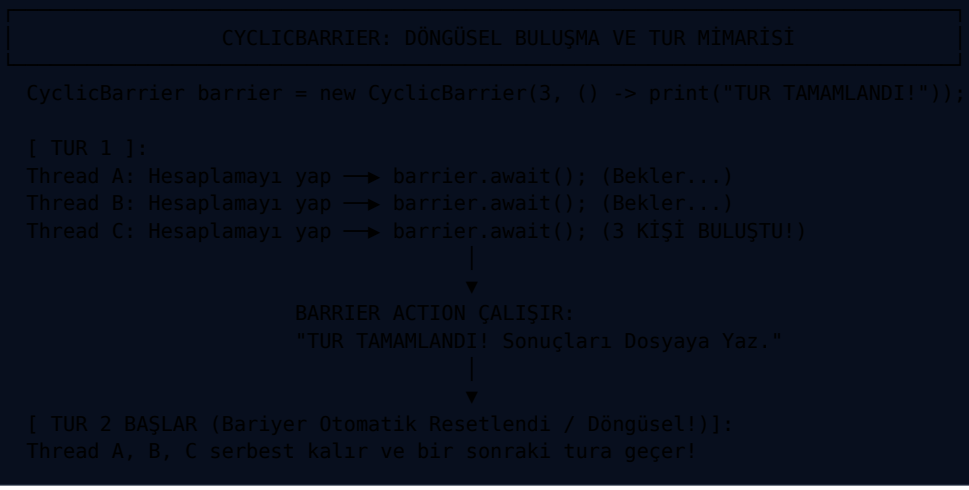
21

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CyclicBarrier: Tekrar Kullanılabilir Buluşma Noktaları & Bariyer Eylemi

CyclicBarrier: Reusable Multi-Thread Rendezvous & Barrier Actions

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunaparktaki 4 kişilik gondol düşünün. Gondol 4 kişi dolmadan ASLA hareket etmez. İlk binen 3 kişi gondolun içinde oturup 4. kişiyi bekler (`await()`). 4. kişi bindiği anda kilitler kapanır ve gondol sallanır (Barrier Action). Tur bitince herkes iner ve gondol TEK-RAR boşalır; sonraki 4 kişi için baştan kullanılır (Cyclic / Döngüsel)! CountDownLatch gibi çöpe atılmaz!

🇬🇧 ENGLISH (ELI5)

Picture a 4-person whitewater rafting crew. The raft cannot depart until all 4 rowers take their seats. The first three climbers sit with paddles ready (`await()`). The instant the fourth rower steps aboard, the team pushes off down the rapids together (Barrier Action). At the bottom of the river, the raft is carried back up and re-used for the next round! Unlike single-use countdown latches, CyclicBarrier automatically resets for subsequent cycles.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CyclicBarrier ve CountDownLatch Farkları: 1) Tekrar Kullanılabilirlik (Cyclic): CyclicBarrier, tüm thread'ler buluştuğunda iç sayacını otomatik olarak ilk değerine geri sıfırlar. Matris çarpımları, genetik algoritmalar ve çok adımlı paralel simülasyonlar için biçilmiş kaftandır. 2) Barrier Action: Tüm thread'ler bariyere ulaştığında, içlerinden rastgele bir tanesi verilen `Runnable` görevini (ör. tur sonuçlarını diskte birleştirme) tek başına çalıştırır; bitince tüm thread'ler bir sonraki adıma serbest bırakılır. 3) Kırık Bariyer (BrokenBarrierException): Eğer bekleyen thread'lerden biri zaman aşımına uğrar veya `interrupt` edilirse, tüm bariyer kırılır (Broken state). Diğer tüm bekleyen thread'ler `BrokenBarrierException` alarak çöker; böylece eksik veriyle yanlış işlem yapılması donanımsal olarak engellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CyclicBarrier Mechanics & Failure Handling: 1) Cyclic Reset: CyclicBarrier resets its internal count automatically once the parties count reaches zero, advancing to a new 'Generation'. It is specifically tuned for iterative algorithms (e.g. parallel Jacobi relaxation or map-reduce matrix passes). 2) Barrier Action Execution: When the last thread trips the barrier, it synchronously executes the barrier action Runnable on its own stack before releasing the awaiting parties. 3) Broken Barriers: If any thread in the barrier is interrupted or times out, the barrier enters the 'Broken' state, unparking all other threads with a `BrokenBarrierException` to prevent deadlock and state corruption.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Çok Turlu Paralel Simülasyonda CyclicBarrier Kullanımı
import java.util.concurrent.CyclicBarrier;

public class ParallelMatrixSimulation {
    void main() {
        int workerCount = 3;

        // 3 işçi buluştuğunda çalışacak ortak bariyer eylemi
        var barrier = new CyclicBarrier(workerCount, () -> {
            IO.println("=== TUR BİTTİ: Ara sonuçlar birleştirildi. Yeni tura geçiliyor! ===\n");
        });

        for (int i = 1; i <= workerCount; i++) {
            final int id = i;
            Thread.ofPlatform().name("Worker-" + id).start(() -> {
                try {
                    for (int round = 1; round <= 2; round++) {
                        IO.println("İşçi " + id + " | Tur " + round + " hesaplıyor...");
                        Thread.sleep(100 * id); // Farklı hızlarda hesaplarlar

                        IO.println("İşçi " + id + " bariyere vardı, diğerlerini bekliyor...");
                        barrier.await(); // 3 kişi tamamlanana kadar bekle
                    }
                } catch (Exception e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CountDownLatch ile CyclicBarrier arasındaki en temel iki fark nedir?' diye sorarsa: '1) CountDownLatch tek kullanımlıktır, CyclicBarrier sonsuz döngüde tekrar kullanılabilir (resetlenir). 2) CountDownLatch thread'lerin bir olayı beklemesini sağlar; CyclicBarrier ise thread'lerin birbirini beklemesini sağlar!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top MAANG Difference Question: 'What are the two foundational differences between CountDownLatch and CyclicBarrier?' Senior Answer: '1) CountDownLatch is single-use and cannot be reset; CyclicBarrier is cyclic and auto-resets per generation. 2) CountDownLatch waits for events/tasks to finish (threads don't wait for each other); CyclicBarrier forces concurrent threads to wait for each other at a mutual rendezvous.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CyclicBarrier her turda kendini sıfırlayarak sonsuz kez tekrar kullanılabilir.
- Barrier Action, son gelen thread tarafından tek başına yürütülür.
- Bir thread kesilirse bariyer Broken state'e geçer ve tüm thread'leri korur.

🎯 Key Takeaways for the Interview

- CyclicBarrier auto-resets its count, enabling iterative multi-phase processing.
- The barrier action is executed by the final arriving thread before unparking the pool.
- Thread interruption breaks the barrier, throwing BrokenBarrierException across all parties.